

Step 02 - Create a Tiny Dataset

A First Practical Machine Learning Workflow

What this document is for

Step 01 got the notebook ready. Step 02 creates the first actual dataset. There is still no model, no training, and no scoring. The whole job here is to make the data shape feel obvious before scikit-learn appears in later steps.

By the end of this step, you should have:

- A small house-price table created in the notebook.
- A clear mental model: each row is one house, and each column is one measured property.
- A separation between input features, called X, and the known answer, called y.
- Shape checks showing that each feature row has one matching target value.

1. Where this step sits

This step sits between environment setup and model training. A model cannot be trained until there is a dataset to train from, and the dataset needs to be understandable before it becomes arrays, splits, and predictions.

Step	What changes
01 - Get the notebook ready	The notebook and package checks exist.
02 - Create a tiny dataset	Examples become rows; features and target are separated.
03 - Split before training	The rows are divided into training and test sets before any fitting happens.

Do not rush past the data

A surprising amount of ML confusion starts with not knowing what a row represents, what a column represents, or which column is the answer. This step is deliberately slow about that.

2. Name the parts before writing code

The toy problem is: predict a house price from a few visible properties of that house. The data is made up, but the table layout is the same layout used by real supervised-learning examples.

Column	Role in this step	Meaning
bedrooms	feature	Input value the model may later use.
floor_area_m2	feature	Input value measured in square metres.
distance_station_km	feature	Input value measured in kilometres.
age_years	feature	Input value measured in years.
price_k	target	Known answer during training, in thousands of pounds.

The key mental model

One row = one house. Most columns are features. The final column, price_k, is the target: the answer we want a later model to learn to predict.

3. Import pandas when the table needs it

The first actual tool needed is pandas. pandas gives us a **DataFrame**, which is a labelled table: rows, columns, column names, and convenient inspection methods.

Why this cell exists

We need a table before we need a model. This import appears here because this is the first moment the notebook needs table functionality.

Code cell

```
import pandas as pd
```

What to look for

The cell should run silently. The short name pd is now available for pandas. Nothing has been created yet.

4. Build the table row by row

The next cell creates two ordinary Python objects first: a list of column names and a list of house rows. Each inner list is one house. The value order follows the column order exactly.

Why this cell exists

This keeps the code aligned with the way you should read the data: one sample at a time. It avoids the more compact pandas style where whole columns are supplied separately, because that can feel sideways when you are learning the workflow.

Code cell

```
columns = [
    "bedrooms",
    "floor_area_m2",
    "distance_station_km",
    "age_years",
    "price_k",
]

house_rows = [
    [1, 45, 0.5, 8, 210],
    [2, 60, 1.2, 30, 245],
    [2, 72, 0.8, 12, 280],
    [3, 80, 2.5, 45, 300],
    [3, 92, 1.0, 20, 335],
    [3, 105, 3.1, 5, 390],
    [4, 110, 1.8, 35, 410],
    [4, 130, 2.2, 15, 455],
    [4, 145, 4.5, 60, 430],
    [5, 155, 2.8, 10, 520],
    [5, 175, 5.0, 75, 500],
    [6, 205, 3.7, 25, 610],
]

houses = pd.DataFrame(house_rows, columns=columns)

houses
```

What to look for

The notebook should display a 12-row table. Read it left to right and row by row: one house, then the next house, then the next. Do not worry yet about whether these numbers are realistic; they exist to make the workflow visible.

5. Check the table shape

Before splitting features from the answer, check that the table has the dimensions you think it has. In pandas, **shape** reports rows first, columns second.

Why this cell exists

Shape checks prevent silent confusion. If you expected 12 houses and 5 columns, the table should say exactly that.

Code cell

```
print("houses shape:", houses.shape)
print("columns:", list(houses.columns))
```

What to look for

The shape should be (12, 5). That means 12 rows and 5 columns. The column list should match the names used when the table was created.

6. Separate the features from the target

Supervised-learning examples usually use **X** for the input features and **y** for the known answer. This is just convention, but it is common enough that it is worth practising.

Why this cell exists

A later model should learn from the input columns, not from the answer column. This cell explicitly separates those two ideas before any training happens.

Code cell

```
feature_columns = [
    "bedrooms",
    "floor_area_m2",
    "distance_station_km",
    "age_years",
]

X = houses[feature_columns]
y = houses["price_k"]
```

What to look for

This cell may not display anything. That is fine. It has created X, the feature table, and y, the target values.

7. Inspect X and y separately

Now display the first few rows of each object. **head()** means "show me the first few rows". It is a quick inspection tool, not a modelling step.

Code cell

```
print("First feature rows:")
print(X.head())

print()
print("First target values:")
print(y.head())
```

What to look for

X should contain the four input columns only. It should not contain price_k. y should contain only the known prices. The first row of X and the first value of y belong together.

8. Confirm that the rows still line up

The feature rows and target values must stay aligned. If row 0 of X describes the first house, value 0 of y must be the price for that same house.

Why this cell exists

ML code often works even when your mental model is wrong. This check is deliberately simple: count the rows and inspect the first row/value pair.

Code cell

```
print("X shape:", X.shape)
print("y shape:", y.shape)
print("Same number of rows:", len(X) == len(y))

print()
print("First feature row:")
print(X.iloc[0])

print()
print("Matching target value:")
print(y.iloc[0])
```

What to look for

X should have shape (12, 4). y should have shape (12,). Same number of rows should be True. The first feature row should describe the one-bedroom, 45-square-metre house, and the matching target value should be 210.

9. Stop before training

It is tempting to train a model immediately, but the next important idea is the train/test split. That must happen before fitting the model, otherwise the evaluation story becomes muddled.

Checkpoint before Step 03

You are ready for Step 03 when you can say: houses is the whole table, X is the feature table, y is the known answer, X has 12 rows and 4 feature columns, and y has 12 matching target values.

Terms to Say Out Loud

Step 02 - Create a Tiny Dataset

Retrieval target

You do not need a dictionary definition. You should be able to say each term in plain English and connect it to the notebook work.

Term	Say it out loud as...
sample	One example row, such as one house.
feature	An input column the model may use.
target	The known answer column the model will later learn to predict.
DataFrame	A labelled pandas table with rows, columns, and column names.
row	One sample in the table.
column	One measured property or value type in the table.
X	The feature table: the inputs.
y	The target values: the known answers.
shape	The row-and-column size of an object, reported as rows first.